

Mastermind — AI Usage

Project 2, Phase 1 · EID: ak54582

1. AI tools used

Tool	Used for
Claude Code (Anthropic), model Claude Opus 5, run as the VS Code extension	Reviewing the finished Phase 1 source; compiling the project and running the test suite to report measured results; drafting the Design, Tests and AI documents, including the class diagram. It did not write or modify any of the <code>assignment2</code> source or test code, which was already complete when the tool was given the project.

No AI tool wrote code that was submitted. The Java sources under `src/assignment2` and `tests/assignment2` are unchanged from the finished Phase 1 project; the tool's role was review, verification and documentation.

2. Important AI interactions

Interaction 1: Deriving the design document from the code

What I asked	Read the Phase 1 project and produce a design write-up: a class/interface diagram, the responsibility of each major type, which pieces are reusable in another game, and which are intentionally Mastermind-specific.
What the AI did	Read all 13 Java files and the README, then drew a UML-style diagram of the 13 types (counting the nested <code>Status</code> enum) showing implements, extends, owns and uses relationships. It color-coded each type as reusable or Mastermind-specific and explained the split at the <code>GuessingGame</code> interface: <code>ConsoleRunner</code> holds only a <code>GuessingGame</code> , so it never sees a <code>peg</code> or a <code>color</code> .
How I checked it	Compared every box and arrow against the source: that <code>AbstractGuessingGame.submit</code> and <code>startNewRound</code> are <code>final</code> , that <code>MastermindGame</code> holds a <code>GameConfiguration</code> and a <code>CodeGenerator</code> , and that <code>Turn</code> stores feedback as a plain <code>String</code> so it carries no Mastermind types.

Interaction 2: Verifying the tests instead of trusting the description

What I asked	Describe at least 8 meaningful tests, covering duplicate-color feedback, invalid input, history, win/loss and a changed configuration.
What the AI did	Rather than summarizing the test file on sight, it compiled the project and ran <code>MastermindTests</code> , which reported 80 passed, 0 failed , and used the real output (including the solver's measured worst case of 9 guesses on secret <code>RYRP</code> , average 5.765) in the write-up. It then selected 14 representative tests and wrote each with setup, input, expected result and rationale. No new tests were invented: every test described already exists in the suite.
How I checked it	Re-derived the duplicate-color answers by hand with the rule "remove exact matches first, then white = $\sum \min(\text{secret count}, \text{guess count})$ per remaining color". For example <code>BBG0</code> vs <code>G BBB</code> : one exact B, leftovers {B, G, O} against {G, B, B}, giving <code>1B_2W</code> . The stated grouping $(13 + 2 + 4 + 10 + 15 + 21 + 9 + 4 + 2)$ also sums to the 80 checks the run reported.

Interaction 3: Getting the documents into PDF without a PDF library

What I asked	Deliver the three documents as PDFs.
What the AI did	Its first approach was to generate the PDFs with Python's <code>reportlab</code> . The install failed on this machine, so instead of stopping it wrote the documents as HTML with a hand-drawn SVG diagram and printed them through headless Microsoft Edge. It then screenshotted the rendered page, noticed that the diagram had been pushed onto page 2 and that browsers drop background colors when printing, and fixed both (a smaller diagram and <code>print-color-adjust: exact</code>).
Why it mattered	This is the interaction where the AI was most useful and least trustworthy at the same time: the first plan simply did not work, and the layout errors were only caught by rendering the result and looking at it. Checking output beat accepting the first answer.

3. An AI-generated code suggestion I rejected

The suggestion. After stress-testing the finished project, the AI reported one rough edge and proposed a code change for it. `GameConfiguration.fromSystemProperties()` reads its overrides with `Integer.getInteger("mastermind.guesses", 12)`. That method returns `null` for a value it cannot parse, so launching with `-Dmastermind.guesses=abc` silently falls back to 12 instead of complaining. The AI suggested adding explicit parsing that rejects a malformed value with a message such as *"Error: mastermind.guesses must be a number"*.

Why I rejected it. The failure is harmless: an unparseable value cannot corrupt a game, produce a wrong score, or crash anything — the program simply runs with the documented default, and the banner states the configuration in use, so the player can see that 12 guesses are in effect. Against that, the change adds two try/catch blocks and a new failure path to the one class whose job is to be a simple, verified holder of settings. The settings that genuinely cannot be defaulted *are* already validated, loudly, in the constructor: 0 pegs, duplicate colors, more than 10 colors and whitespace colors all abort with a clear `Error`: line and exit code 1. I would rather keep that one validation path than split it across two mechanisms for a typo the player

can already see the effect of. If Phase II makes configuration user-facing rather than a developer flag, I would revisit it.

This is also the general point about the tool: it was reliable at finding things to check, and wrong about whether one of them was worth changing. The measurements it produced were useful; the conclusion it drew from them was mine to make.

4. A design decision I reconsidered

Separately from the AI's suggestions, the design decision I reconsidered during development was **who starts the first round**, which the test suite still guards with a regression test.

First approach. `MastermindGame`'s constructor calls `startNewRound()`, so a freshly constructed game is immediately playable through the API and no caller can forget to set it up. The natural way to write the console loop is then to call `game.startNewRound()` at the top of every round, including the first.

The problem. Those two choices quietly conflict. With random secrets nothing looks wrong, because one random code is as good as another. With `FixedCodeGenerator("BGOP", "YYYY")` in testing mode, though, the constructor consumes `BGOP` and the loop immediately discards it and draws `YYYY`. The first secret the tester asked for is silently skipped, and a single-secret list still appears to work only because the generator cycles. This is exactly the kind of bug that makes a testing mode untrustworthy: the person testing cannot tell whether the game or their own guess was wrong.

Alternatives considered. (a) Remove the constructor's call, so every caller must start a round before playing. That makes the API easier to misuse and pushes an extra line into every test that builds a game directly. (b) Keep the constructor's call and let the runner start a new round only *between* rounds.

What I changed to. Option (b). The contract is now stated in the `GuessingGame.startNewRound()` documentation: "a newly constructed game is already ready to play; call this only to start a further round". `ConsoleRunner.run()` plays the first round as constructed, then calls `startNewRound()` only after the player answers Y at the play-again prompt. Because this is an invisible failure, a named regression test pins it: "*first secret from a fixed list is used first (not skipped)*" runs two rounds in test mode with fixed secrets `BGOP` and `YYYY` and asserts that `Secret: BGOP` is printed before `Secret: YYYY`.

A second option I turned down: a generic secret source

While arranging the code for reuse, the tempting next step was to make `CodeGenerator` generic — something like `SecretSource<S>` — so a future Wordle or Hangman implementation could reuse it alongside `GuessingGame` and `AbstractGuessingGame`. I rejected it for Phase 1. The interface takes a `GameConfiguration`, which is entirely about pegs and colors, so making it generic would either drag that `Mastermind` type into the reusable layer or add type parameters to `MastermindGame` that no second game yet justifies. With only one game there is nothing to design the abstraction against, and a guessed abstraction is usually the wrong one. `CodeGenerator` therefore stays `Mastermind`-specific, and `Design.pdf` records it as the first refactor to make when a second game actually arrives — at which point the real requirements are known.